

Real-world (how long can this conversation actually go?) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (the real formula, with a safety margin built in). Read Basic first even if you're comfortable with the math — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the formula](#)
- [Hint 2 — A worked example, and the trimming decision](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the formula

Basic Version

Think of the context window as one shared bucket, not a separate bucket for "what I send" and "what I get back." Every message ever sent in the conversation — yours and the model's — sits in that same bucket, because the whole history gets resent every time.

So the question "how many turns fit" is really: how big is the bucket, and how much does one back-and-forth turn (your message + the model's reply) add to it?

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The context window is a fixed token budget for the whole call, shared by the system prompt, every prior message in the resent history, and the room the model needs left over to write its reply. Nothing about the window itself distinguishes "old messages" from "new reply" — it's one number, consumed from all sides.

Build the estimate as a small budget, not one giant division:

- Reserve tokens for the system prompt (fixed, paid every single call).
- Reserve tokens for the model's own reply room (budget generously — replies often run longer than the user's message).
- Whatever's left over is what turns get to share.

```
turns_that_fit = (window_size - system_prompt_tokens) / average_tokens_per_turn
```

Pick a window size and an average turn size before moving to Hint 2.

Keep in mind the formula assumes every turn is the same size — real conversations never are. A support chat has short turns throughout. A document-Q&A feature has one huge turn (the pasted document) followed by short follow-ups. A safety margin (start trimming at 75% full, not 100%) is what actually protects you from that unevenness, not a more precise average.

Difference between Basic and Intermediate: Basic names the idea (one shared bucket) and the two things you need to know (window size, per-turn cost). Intermediate turns that into an actual formula with a concrete budget breakdown and a safety margin.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — A worked example, and the trimming decision

Basic Version

```
window = 128,000 tokens
system prompt = 100 tokens
one turn (user message + reply) = 100 tokens

leftover = 128,000 - 100 = 127,900
turns that fit = 127,900 / 100 = about 1,279 turns
```

That's a lot of turns for a short-message chat — the number gets much smaller fast once messages get longer (pasted documents, long code blocks).

Once you've got a number, ask the follow-up question this exercise is really testing: what should your program actually *do* long before that number of turns is reached? Waiting until the window is full and the call just fails is the wrong answer.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
window_size = 128_000
system_prompt_tokens = 100
avg_user_message_tokens = 50
# replies tend to run longer than a short user message
avg_reply_tokens = 300
avg_turn_tokens = avg_user_message_tokens + avg_reply_tokens    # 350

leftover = window_size - system_prompt_tokens                  # 127,900
turns_that_fit = leftover / avg_turn_tokens    # about 365 turns
```

Notice how much the estimate moved (1,279 turns down to about 365) just from being honest that replies usually run longer than the user's own message.

A common real-world rule is to start trimming or summarizing older messages once you've used somewhere around 70-80% of the window, not 100% — that leaves headroom for an unusually long reply and avoids a hard failure appearing suddenly mid-conversation.

Two design details worth writing down alongside your number, for when this becomes real code: never trim the system prompt (it's not part of history, it's a fixed instruction), and never trim the most recent turn or two (the model needs the immediate context to make sense of a follow-up question like "what about the second one?").

Write your final turn estimate, the percentage where you'd start trimming, and these two rules, then compare against the [Solution](#).

Difference between Basic and Intermediate: Basic gets a single static number from a single static formula. Intermediate refines the formula's inputs (realistic reply length, a safety margin instead of 100%) and adds the two rules a formula alone doesn't tell you.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)